

My LADMM, Reordered, Is Adaptive PDHG

Optimal Transport / Schrödinger Bridge Project

Companion to `nonuniform_grid_xy_variables.tex`, `nonuniform_grid_norms.tex`

Abstract

Written from the top, result first rather than reconstructed at the end. `ladmm.py`'s iteration, *reordered to begin at its second step* – the y -update rather than the x -update – is exactly Goldstein–Li–Yuan's PDHG (their PDHG step, eqs (2)–(5)), in their own ordering, term for term and index for index, for *any* step-size sequence γ_k, τ_k , adaptive or not. Nothing is linearized differently, no variable is added, no step is lagged; only the point at which one iteration is declared to end moves. Consequently their adaptive step-size rule (their Algorithm 1) transfers verbatim, together with its convergence proof, and their stability condition turns out to be the $\tau > \gamma \|A\|^2$ this project already imposes. Part I states the reordered scheme, Part II derives the saddle point it is run on and states their Algorithms 1 and 2 there, Part III matches the two line by line, Part IV collects the checks, Part V writes out the resulting adaptive LADMM in full, and Part VI asks what the adaptive-LADMM literature actually runs (answer: the other cut, which is a genuinely different algorithm once the step size moves).

Part I – my LADMM, reordered to start at the y -update

Iterates x^k, y^k, δ^k ; scalars $\gamma_k > 0$ (penalty) and $\tau_k > 0$ (proximal step), iteration-dependent from the start since the whole point is to let them adapt; linear/affine map A (boundary conditions baked in), constant d , adjoint $A^\top$ (the *linear part only* – no constant); convex f_1, f_2 . The constraint is $Ax - y - d = 0$, i.e. `ladmm.py`'s $Ax + By - b$ at $B = -I, b = d$.

One cycle, $k \rightarrow k+1$, carrying (x^k, δ^k) in and (x^{k+1}, δ^{k+1}) out, with the *same* γ_k, τ_k on every line:

$$y^{k+1} := \text{prox}_{f_2/\gamma_k} (Ax^k - \delta^k/\gamma_k) \quad (1)$$

$$r^k := Ax^k - y^{k+1} - d \quad (2)$$

$$\delta^{k+1} := \delta^k - \gamma_k r^k \quad (3)$$

$$x^{k+1} := \text{prox}_{(1/\tau_k)f_1} \left(x^k - \frac{1}{\tau_k} A^\top (\gamma_k r^k - \delta^{k+1}) \right) \quad (4)$$

where $\text{prox}_{th}(z) := \arg \min_p h(p) + \frac{1}{2t} \|p - z\|^2$.

This is the same scheme, cut in a different place. `ladmm.py` runs the cycle (r, x, y, δ) : it computes its residual from the *stored* y , updates x , then updates y and δ . Above, the cycle is (y, r, δ, x) – the same four operations, in the same order along the same tape of updates, with the boundary between iterations moved from after the δ -update to after the x -update. Unrolled, both groupings read off the one sequence

$$\dots \mid y^{k+1}, \delta^{k+1} \mid x^{k+1} \mid y^{k+2}, \delta^{k+2} \mid x^{k+2} \mid \dots$$

and differ only in where the bars go. What the choice of cut buys is stated in Part III; the short version is that this cut puts the δ -update and the x -update that consumes $\gamma_k r^k$ inside the *same* cycle, so both refer to the same γ_k by construction and no ratio of consecutive penalties can appear.

The residual carries mixed indices, necessarily. (2) is $r^k = Ax^k - y^{k+1} - d$, not $Ax^k - y^k - d$: starting the cycle at the y -update means the y that r^k measures against is the one *this* cycle has just produced, while the x is still the incoming one. Both indices are correct as written – r^k is a genuine constraint residual, just not at a single index. (Its single-index cousin $D^{k+1} := Ax^{k+1} - y^{k+1} - d$, `ladmm.py`'s own constraint violation, reappears on its own in Part III.)

Boundary convention, spelled out rather than left implicit. A cycle of (1)–(4) needs only (x^0, δ^0) and manufactures its own y^1 from them in (1). With $\delta^0 := 0$ (`ladmm.py`'s own initialization) that is the entire starting state: no y^0 is required anywhere – one fewer arbitrary initial value than the unrotated cut, whose given y^0 is consumed by its very first x -update and never re-derived.

Where the constant d has to live. Worth flagging, because it is the one place the reordering is not indifferent to notation. (1) evaluates $\text{prox}_{f_2/\gamma_k}$ at $Ax^k - \delta^k/\gamma_k$ while (2) subtracts d ; if d is carried loose in the residual like that and *not* inside the map the y -update sees, then Part III's dual collapse acquires a stray $+\gamma_k d$ and the correspondence fails by exactly that much (checked numerically – the error is $\gamma_k d$ on the nose). The fix is to read A throughout (1)–(4) as the affine map with d folded in, $Ax \mapsto Ax - d$, which is precisely what `ladmm.py`'s `A_fn` already is: affine, boundary data baked in, and called with $b = 0$ in every run of this project, so d never appears loose and the point is moot in practice. $A^\top$ (`At_fn`) stays the linear part with no constant – the same asymmetry `pdhg.py`'s docstring warns about, and the reason differences of A must be formed by evaluating it twice rather than once on a difference. Everything below is written for A affine in this sense.

Two simplifications, both flagged. First, $\text{prox}_{(1/\tau_k)f_1}$ genuinely depends on τ_k in general – this is the standard proximal-gradient step, step $1/\tau_k$ on both the gradient move and the prox – and everything below is written to hold in that generality. In *this* project's problem f_1 is the FP-feasibility indicator, so $\text{prox}_{(1/\tau_k)f_1}$ collapses to the plain projection for any τ_k ; nothing below relies on that coincidence. Second, `ladmm.py` carries an over-relaxation parameter α (replacing Ax^k by $\alpha Ax^k + (1 - \alpha)(d + y^k)$ in (1)–(3)); every run in this project uses $\alpha = 1$, which is the substitution already made above, so (1)–(4) is the iteration actually being run.

Part II – Goldstein–Li–Yuan's PDHG, on this problem's saddle point

Their scheme, in their own layout, but instantiated on the saddle point it will turn out to be run on rather than left abstract. Symbol discipline as before: no letter is reused from Part I. A , f_1 , f_2 are reused deliberately – they *are* Part I's, not analogues – and everything else gets a fresh letter.

Which saddle point

Part I's constrained form $\min f_1(x) + f_2(y)$ s.t. $Ax - y - d = 0$ is, with d folded into A (Part I),

$$\min_x f_1(x) + f_2(Ax).$$

Its Fenchel dual, by the standard formula ($\min f(\cdot) + g(K\cdot)$ has dual $\min f^*(-K^\top \cdot) + g^*(\cdot)$), is $\min_v f_2^*(v) + f_1^*(-A^\top v)$. Legendre-transform *that* problem's composed piece, using $(f_1^*)^* = f_1$, so $f_1^*(-A^\top v) = \max_u \langle -A^\top v, u \rangle - f_1(u)$:

$$\min_v \max_u f_2^*(v) + \langle Kv, u \rangle - f_1(u), \quad K := -A^\top. \quad (5)$$

Set against Goldstein–Li–Yuan's own template $\min \max F(\cdot) + \langle K\cdot, \cdot \rangle - G(\cdot)$, this fixes

$$\boxed{F = f_2^*, \quad G = f_1, \quad K = -A^\top}$$

Worth naming the reversal explicitly, since it inverts the obvious guess. Read the *primal* problem in the usual Chambolle–Pock way, $\min f(\cdot) + g(K\cdot)$ with $f = f_1$ and $g = f_2$, and one expects $F = f$, $G = g^*$. Here it is the other way round: $F = g^* = f_2^*$ and $G = f = f_1$, with the operator $-A^\top$ rather than A . So the function on the variable that updates *first* is the conjugate of Part I's f_2 , and the one on the variable that updates *second* is Part I's f_1 untouched. Part III shows which of Part I's iterates each of v, u is; the answer is likewise the reverse of the obvious guess.

Renaming table

Their symbol	What it is	Written here as
x (iterate updated first)	clashes with Part I's x	v
y (iterate updated second)	clashes with Part I's y	u
f (function on the first)	$= F = g^* = f_2^*$	f_2^*
g (function on the second)	$= G = f = f_1$	f_1
A (coupling operator)	$= -A^\top$, <i>not</i> Part I's A	K
τ_k (step in the first update)	clashes with Part I's τ	θ_k
σ_k (step in the second update)	no clash	σ_k
α_k (adaptivity level)	clashes with Part I's α	ω_k
p_k (primal residual)	no clash	p_k
d_k (dual residual)	clashes with Part I's constant d	q_k
$\rho(\cdot)$ (spectral radius)	clashes with ρ_k , Part VI	$\lambda_{\max}(\cdot)$
η (adaptivity decay)	no clash	unchanged

The PDHG step (their (2)–(5))

Their five lines, verbatim in structure, on (5). Steps $\theta_k, \sigma_k > 0$; the right-hand column substitutes $K = -A^\top$ (so $K^\top = -A$) to show the same line in Part I's own operator:

$$\hat{v}^{k+1} = v^k - \theta_k K^\top u^k = v^k + \theta_k A u^k \quad (6)$$

$$v^{k+1} = \arg \min_v f_2^*(v) + \frac{1}{2\theta_k} \|v - \hat{v}^{k+1}\|^2 = \text{prox}_{\theta_k f_2^*}(\hat{v}^{k+1}) \quad (7)$$

$$\bar{v}^{k+1} = v^{k+1} + (v^{k+1} - v^k) = 2v^{k+1} - v^k \quad (8)$$

$$\hat{u}^{k+1} = u^k + \sigma_k K \bar{v}^{k+1} = u^k - \sigma_k A^\top \bar{v}^{k+1} \quad (9)$$

$$u^{k+1} = \arg \min_u f_1(u) + \frac{1}{2\sigma_k} \|u - \hat{u}^{k+1}\|^2 = \text{prox}_{\sigma_k f_1}(\hat{u}^{k+1}) \quad (10)$$

Collapsing the intermediates, the two lines that matter are

$$v^{k+1} = \text{prox}_{\theta_k f_2^*}(v^k + \theta_k A u^k) \quad (11)$$

$$u^{k+1} = \text{prox}_{\sigma_k f_1}(u^k - \sigma_k A^\top (2v^{k+1} - v^k)) \quad (12)$$

The ordering is the whole point and is worth stating in words: v updates first from the *stored* u and takes no extrapolation; u updates second from the *freshly computed and extrapolated* v . The extrapolation coefficients are the literal, fixed (2, -1) – (8) is their own $\bar{v}^{k+1} = v^{k+1} + (v^{k+1} - v^k)$, not a generalization of it.

The adaptive rule (their Algorithm 1, residual balancing)

After each PDHG step, form the two residuals (again with $K = -A^\top$ substituted on the right):

$$p_{k+1} = \left| \frac{1}{\theta_k} (v^k - v^{k+1}) - K^\top (u^k - u^{k+1}) \right| = \left| \frac{1}{\theta_k} (v^k - v^{k+1}) + A(u^k - u^{k+1}) \right| \quad (13)$$

$$q_{k+1} = \left| \frac{1}{\sigma_k} (u^k - u^{k+1}) - K(v^k - v^{k+1}) \right| = \left| \frac{1}{\sigma_k} (u^k - u^{k+1}) + A^\top (v^k - v^{k+1}) \right| \quad (14)$$

then adapt, with adaptivity level ω_k and their hard-coded imbalance factor 2:

$$p_{k+1} > 2q_{k+1} \Rightarrow \theta_{k+1} = \frac{\theta_k}{1-\omega_k}, \quad \sigma_{k+1} = \sigma_k(1-\omega_k), \quad \omega_{k+1} = \eta\omega_k;$$

$$2p_{k+1} < q_{k+1} \Rightarrow \theta_{k+1} = \theta_k(1-\omega_k), \quad \sigma_{k+1} = \frac{\sigma_k}{1-\omega_k}, \quad \omega_{k+1} = \eta\omega_k;$$

otherwise $\theta_{k+1} = \theta_k$, $\sigma_{k+1} = \sigma_k$, $\omega_{k+1} = \omega_k$. Their constants: $\omega_0 = \eta = 0.95$.

Two deviations from their line 1, stated rather than buried. First, their line 1 chooses *deliberately large* θ_0, σ_0 and relies on a backtracking check (their line 4, halving both steps when it fails) to pull them back into the stable region. That backtracking step is **not** transcribed here and **not** implemented in `ladmm_reordered.py`. Instead we start inside the stable region, $\theta_0 \sigma_0 < \lambda_{\max}(K^\top K)^{-1}$ with $K^\top K = AA^\top$ – legitimate precisely because the product is invariant under the rule (next paragraph), so a start inside the region stays inside it forever and the backtracking branch could never fire. Second, their $\omega_0 = 0.95$ multiplies a step by $1/(1-\omega_0) = 20$ on its first firing; that aggressiveness is presumably calibrated against the large- θ_0, σ_0 start it is paired with, and is worth re-examining here, where it lands on an already-stable starting point (Part V, and the ringing noted there).

Source note, since two papers are easy to confuse. The constants above – $\omega_0 = \eta = 0.95$ and the hard-coded factor 2 – are Goldstein–Li–Yuan’s own, from the three-author paper’s Algorithm 1. They are *not* the ones in the longer five-author technical report (Goldstein, Li, Yuan, Esser & Baraniuk, arXiv:1305.0546), whose Algorithm 2 is the same rule with $\omega_0 = 0.5$, an imbalance threshold $\Delta = 1.5$, and an extra residual pre-scale s that does not appear here at all. The two are different write-ups of related schemes; their constants must not be mixed.

Two structural remarks, both used in Part IV. Each branch multiplies one step by $(1 - \omega_k)$ and divides the other by it, so the product $\theta_k \sigma_k$ is *invariant* – the rule cannot break the requirement it started inside, and equally cannot tune the product. And ω_k decays geometrically by η , so the adaptation switches itself off; that is what makes the perturbation summable and the convergence proof go through.

Part III – relating them

Part II fixed F, G, K from (5). All that is left is to say which of Part I’s quantities the iterates and steps are, and check the two lines. Two substitutions and two identifications:

$$v^k := -\delta^k, \quad u^k := x^k, \quad \theta_k := \gamma_k, \quad \sigma_k := \frac{1}{\tau_k}.$$

Both are again the reverse of the obvious guess: their first-updated iterate is Part I’s *multiplier*, not its x ; and their first step is Part I’s *penalty* γ , while their second is $1/\tau$.

Line (11): the y - and δ -updates collapse to one prox

(1) and (3) are two steps in Part I but a single prox in the PDHG step, and Moreau’s identity is what merges them. For $h := f_2^*$, $t := \gamma_k$ it reads $\text{prox}_{\gamma_k f_2^*}(\zeta) = \zeta - \gamma_k \text{prox}_{f_2/\gamma_k}(\zeta/\gamma_k)$ (verified numerically to machine precision in an earlier pass). Evaluate at $\zeta := \gamma_k A x^k - \delta^k$, chosen so that $\zeta/\gamma_k = A x^k - \delta^k/\gamma_k$ is exactly (1)’s argument and hence $\text{prox}_{f_2/\gamma_k}(\zeta/\gamma_k) = y^{k+1}$:

$$\text{prox}_{\gamma_k f_2^*}(\gamma_k A x^k - \delta^k) = \gamma_k (A x^k - y^{k+1}) - \delta^k \stackrel{(2)}{=} \gamma_k r^k - \delta^k \stackrel{(3)}{=} -\delta^{k+1}.$$

In the new letters the left side is $\text{prox}_{\theta_k f_2^*}(v^k + \theta_k A u^k)$ and the right side is v^{k+1} , i.e. exactly (11). ✓

(The middle equality is where d must already sit inside A – see Part I. With d carried loose, $A x^k - y^{k+1} = r^k + d$ and the line ends at $-\delta^{k+1} + \gamma_k d$ instead.)

Line (12): the x -update’s bracket is the fixed $(2, -1)$

This is the step the choice of cut was made for. Because (3) and (4) sit in the same cycle they refer to the same γ_k , so (3) rearranges to $\gamma_k r^k = \delta^k - \delta^{k+1}$ with no ratio of consecutive penalties anywhere, and (4)’s bracket is

$$\gamma_k r^k - \delta^{k+1} = (\delta^k - \delta^{k+1}) - \delta^{k+1} = \delta^k - 2\delta^{k+1} \stackrel{\delta=-v}{=} 2v^{k+1} - v^k \stackrel{(8)}{=} \bar{v}^{k+1}.$$

So (4) reads $u^{k+1} = \text{prox}_{(1/\tau_k)f_1}(u^k - \frac{1}{\tau_k} A^\top \bar{v}^{k+1})$, which is (12) with $\sigma_k = 1/\tau_k$. ✓

Note what this did *not* take: no lag, no auxiliary variable, no assumption that γ_k is constant, and no generalized extrapolation coefficient. Part I’s own bracket *is* Goldstein–Li–Yuan’s $\bar{v}^{k+1}$, line (8), unmodified.

An exact equivalence

$$\boxed{v^k = -\delta^k, \quad u^k = x^k, \quad K = -A^\top, \quad F = f_2^*, \quad G = f_1, \quad \theta_k = \gamma_k, \quad \sigma_k = \frac{1}{\tau_k}}$$

Part I *is* the PDHG step – both lines, unmodified template, fixed coefficients, no index offset between the two variables, for any γ_k, τ_k . Their adaptive rule’s updates therefore translate directly into $\gamma_{k+1} := \theta_{k+1}$ and $\tau_{k+1} := 1/\sigma_{k+1}$, with nothing extra to carry along.

Where did y^k go?

Not lost – Moreau’s identity above only *needed* the combination $\gamma_k(Ax^k - y^{k+1}) - \delta^k$, but it computed it by evaluating $\text{prox}_{f_2/\gamma_k}$ along the way, and that inner evaluation *is* y^{k+1} ((1), verbatim). Reading it back off from $\text{prox}_{\gamma_k f_2^*}(\zeta) = \zeta - \gamma_k y^{k+1} = -\delta^{k+1}$ at $\zeta = \gamma_k Ax^k - \delta^k$:

$$y^{k+1} = Ax^k + \frac{\delta^{k+1} - \delta^k}{\gamma_k} = Au^k + \frac{v^k - v^{k+1}}{\theta_k} \quad (15)$$

so y is recoverable from (v, u) at every iteration and needs no separate storage – which is why `pdhg.py` can return this iteration’s KE-optimal state as a by-product of its Moreau step rather than recomputing it.

What the adaptive rule’s residuals are, in Part I’s own quantities

Substitute the dictionary into (13), using $\frac{1}{\gamma_k}(\delta^{k+1} - \delta^k) = -r^k$:

$$p_{k+1} = \left| \frac{1}{\gamma_k}(\delta^{k+1} - \delta^k) + A(x^k - x^{k+1}) \right| = \left| -r^k + Ax^k - Ax^{k+1} \right| = \left| Ax^{k+1} - y^{k+1} - d \right| = \left| D^{k+1} \right|,$$

the constraint violation D^{k+1} `ladmm.py` already computes and stores. The affine constant cancels because only differences of A enter. Likewise (14), using $v^k - v^{k+1} = \delta^{k+1} - \delta^k = -\gamma_k r^k$, becomes

$$q_{k+1} = \left| \tau_k(x^k - x^{k+1}) - \gamma_k A^\top r^k \right|,$$

built from quantities (2)–(4) compute anyway. Both identities were checked numerically (Part IV). So the rule driven by p, q is driven by $\|D^{k+1}\|$ against a τ, γ -weighted measure of the x -step – not the same expression as `ladmm.py`’s own stored P^k , which is worth not conflating, but the same two ingredients.

Part IV – checks, and what stays open

Stability, recovered rather than assumed. The rule needs $\theta_k \sigma_k < \lambda_{\max}(K^\top K)^{-1}$ (Part II). Here $K^\top K = AA^\top$ and $\theta_k \sigma_k = \gamma_k / \tau_k$, so the condition reads

$$\frac{\gamma_k}{\tau_k} \lambda_{\max}(AA^\top) < 1 \quad \Longleftrightarrow \quad \tau_k > \gamma_k \|A\|^2,$$

exactly the convergence condition `ladmm.py` states for itself. Two consequences. Since $\|A\|_N^2 \leq 1$ on any grid for this problem (`nonuniform_grid_norms.tex` Sec 6), $\tau_k > \gamma_k$ suffices – the $\tau = 101\gamma/100$ the configs use. And since each Algorithm-2 branch preserves $\theta_k \sigma_k$, it preserves γ_k / τ_k : the rule moves γ and τ by the *same* factor and can never cross the stability boundary. That is the same shape as `ladmm.py`’s `BoydAdaptiveStepSize`, which rescales both together for exactly this reason; their rule differs in the trigger and in using a decaying factor $1/(1 - \omega_k)$ rather than a fixed one. It also means the rule tunes only the common scale of (γ, τ) , never their ratio.

Causality, and the matched pair. The rule forms $\theta_{k+1}, \sigma_{k+1}$ – that is γ_{k+1} and τ_{k+1} – from p_{k+1}, q_{k+1} , which depend on $v^k, v^{k+1}, u^k, u^{k+1}$, i.e. on $\delta^k, \delta^{k+1}, x^k, x^{k+1}$. All four are in hand the moment cycle k finishes, and cycle $k+1$ does not need γ_{k+1} until its opening y -update. So the two step sizes come out of the same residual pair and govern the same cycle, which is what the rule assumes of its own (θ, σ) .

Checked numerically. On a toy f_1, f_2, A with genuinely varying γ_k, τ_k , a nonzero constant d folded into A , and f_1 a quadratic rather than an indicator – so that $\text{prox}_{(1/\tau_k)f_1}$ really does depend on its step, exercising the $\sigma_k = 1/\tau_k$ identification and not just the gradient coefficient – all of the following hold to machine precision: the bracket collapse of Part III; the Moreau collapse (and its failure by exactly $\gamma_k d$ when d is left loose); (11)–(12) reproducing (1)–(4) iterate for iterate with no index offset; and both residual identities $p_{k+1} = |D^{k+1}|$, $q_{k+1} = |\tau_k(x^k - x^{k+1}) - \gamma_k A^\top r^k|$.

What is not claimed. Two things. First, the reordered cycle and `ladmm.py` as it currently stands are the same sequence of iterates only when γ is constant; once γ moves they differ (Part VI), so this is a statement about which adaptive LADMM to run, not a re-description of the current one. Second, whether their constants ($\omega_0 = \eta = 0.95$, factor 2) are good *here* is an empirical question this note does not touch; and Goldstein–Li–Yuan’s backtracking variant, which is their only mechanism that changes $\theta_k \sigma_k$ and hence the one thing that could tune γ/τ , is not transcribed above.

Part V – adaptive LADMM, written out

Everything needed is now fixed: Part III’s dictionary says $\theta_k = \gamma_k$ and $\sigma_k = 1/\tau_k$, and gives the rule’s residuals in Part I’s own quantities. Substituting turns it into a rule on (γ_k, τ_k) with no PDHG left in it.

Translating the step-size rule

Take their first branch, $\theta_{k+1} = \theta_k/(1 - \omega_k)$ and $\sigma_{k+1} = \sigma_k(1 - \omega_k)$, and read it through the dictionary:

$$\gamma_{k+1} = \theta_{k+1} = \frac{\gamma_k}{1 - \omega_k}, \quad \frac{1}{\tau_{k+1}} = \sigma_{k+1} = \frac{1 - \omega_k}{\tau_k} \implies \tau_{k+1} = \frac{\tau_k}{1 - \omega_k}.$$

The second branch gives $\gamma_{k+1} = \gamma_k(1 - \omega_k)$ and $\tau_{k+1} = \tau_k(1 - \omega_k)$ the same way. So although their rule moves θ and σ in *opposite* directions, in LADMM’s variables it scales γ and τ by *the same factor* – because σ is $1/\tau$, not τ . That single observation is what makes the rule usable here, and it is worth stating separately (next subsection).

The algorithm

Choose x^0, δ^0 (e.g. 0), initial steps $\gamma_0, \tau_0 > 0$ with $\tau_0 > \gamma_0 \|A\|^2$, and their constants $\omega_0, \eta \in (0, 1)$ (their Algorithm 1 line 1 sets $\omega_0 = \eta = 0.95$). For $k = 0, 1, 2, \dots$:

- (1) $y^{k+1} = \text{prox}_{f_2/\gamma_k}(Ax^k - \delta^k/\gamma_k)$
- (2) $r^k = Ax^k - y^{k+1} - d$

$$(3) \quad \delta^{k+1} = \delta^k - \gamma_k r^k$$

$$(4) \quad x^{k+1} = \text{prox}_{(1/\tau_k)f_1} \left(x^k - \frac{1}{\tau_k} A^\top (\delta^k - 2\delta^{k+1}) \right)$$

(5) residuals, from quantities (1)–(4) already produced:

$$p_{k+1} = \|Ax^{k+1} - y^{k+1} - d\| = \|D^{k+1}\|, \quad q_{k+1} = \|\tau_k(x^k - x^{k+1}) - \gamma_k A^\top r^k\|$$

(6) adapt, by the *same* factor on both steps:

$$p_{k+1} > 2q_{k+1} : \quad \gamma_{k+1} = \frac{\gamma_k}{1 - \omega_k}, \quad \tau_{k+1} = \frac{\tau_k}{1 - \omega_k}, \quad \omega_{k+1} = \eta \omega_k$$

$$2p_{k+1} < q_{k+1} : \quad \gamma_{k+1} = \gamma_k(1 - \omega_k), \quad \tau_{k+1} = \tau_k(1 - \omega_k), \quad \omega_{k+1} = \eta \omega_k$$

$$\text{otherwise:} \quad \gamma_{k+1} = \gamma_k, \quad \tau_{k+1} = \tau_k, \quad \omega_{k+1} = \omega_k$$

Step (4) is (4) with its bracket already collapsed ($\gamma_k r^k - \delta^{k+1} = \delta^k - 2\delta^{k+1}$, Part III), which is both cheaper and the form that makes the extrapolation visible. Steps (1)–(4) are Part I verbatim; only (5)–(6) are new.

What the translation buys

The stability margin is preserved for free. Both branches multiply γ_k and τ_k by the same number, so

$$\frac{\gamma_k}{\tau_k} = \frac{\gamma_0}{\tau_0} \quad \text{for every } k,$$

and $\tau_k > \gamma_k \|A\|^2$ holds at every iteration if and only if it holds at $k = 0$. No per-iteration check, no backtracking, no re-estimate of $\|A\|$. This is the LADMM reading of Part IV’s observation that the rule leaves $\theta_k \sigma_k$ invariant – and it is exactly the property `ladmm.py`’s `BoydAdaptiveStepSize` was hand-built to have, here arriving as a consequence rather than a design choice. With $\|A\|^2 \leq 1$ on this problem, $\tau_0 > \gamma_0$ suffices, e.g. the $\tau_0 = 1.01\gamma_0$ the configs already use.

δ needs no rescaling when γ changes. A live worry in `ladmm.py`’s docstring (“no rescaling of delta when gamma changes”). The correspondence settles it: the PDHG iterate is $v^k = -\delta^k$ with *no* γ in the relation, and the rule simply hands the next iteration a new θ_{k+1} without transforming v . So not rescaling δ is correct, not an approximation. (The worry is a real one for *scaled*-form ADMM, where the stored dual is δ/γ and a change in γ does move it; this formulation stores the unscaled δ .)

What it still cannot do. Since γ_k/τ_k is frozen at its initial value, the rule tunes only the common scale of the two steps, never their ratio. A badly chosen ratio – τ_0 far above $\gamma_0\|A\|^2$, i.e. over-damped – stays badly chosen. Changing it needs a mechanism that moves $\theta_k \sigma_k$, which in Goldstein–Li–Yuan is the backtracking variant, not transcribed here.

Against `ladmm.py`'s current rule – a direction to check

`BoydAdaptiveStepSize` has the same shape (rescale τ and γ together) but triggers the opposite way. Written against this module's own stored histories, it does

$$P^k > \mu D^k \Rightarrow \gamma, \tau \text{ both } \times \text{ factor}; \quad D^k > \mu P^k \Rightarrow \gamma, \tau \text{ both } \div \text{ factor},$$

i.e. it *increases* the steps when D^k – the constraint violation – is *small* relative to P^k . The rule derived above does the reverse: $p_{k+1} = \|D^{k+1}\|$ large is precisely the branch that increases γ and τ . Boyd et al. (2011, §3.4.1) themselves increase the penalty when the *primal* residual (the constraint violation) dominates, agreeing with the translated rule and not with the code.

Flagged rather than asserted, because the two “dual residuals” are not the same expression – q_{k+1} above is $\tau_k(x^k - x^{k+1}) - \gamma_k A^\top r^k$, while `ladmm.py`'s P^k is $\tau(x^{k-1} - x^k) + \gamma A^\top (D^{k-1} - D^k)$ – so this is not literally the same test. But the plain reading is that `ladmm.py`'s module docstring names D the “dual residual” and P the “primal residual”, inverted relative to Boyd's own convention ($D = Ax + By - b$ is Boyd's *primal* residual), and that the inversion propagated into the direction of the rule. Worth checking against a run before trusting either.

Checked numerically

The algorithm above was run against the PDHG step + adaptive rule (Part II) on the toy problem of Part IV, from the same start, to a 10^{-12} residual tolerance. Both stop at the same iteration (66), with the adaptation firing on 10 of them and sweeping γ over a factor of about 72; x^k and δ^k agree with u^k and $-v^k$ to machine precision throughout, γ_k matches θ_k to *exactly* zero, and γ_k/τ_k is constant to 15 digits. Past convergence the two part ways harmlessly: with p_{k+1}, q_{k+1} both at rounding-noise level ($\sim 10^{-16}$), the branch test in (13)–(14) is decided by rounding, and the two algebraically-equal residual formulas round differently. Any implementation with a stopping rule never reaches that regime – but it is a reason not to keep adapting after the residuals stop being meaningful.

Part VI – what does the adaptive-LADMM literature actually do?

Checked (web search, not exhaustive) rather than assumed. Two representative papers that adapt a linearized-ADMM-style penalty:

- Lin, Liu & Su, “Linearized Alternating Direction Method with Adaptive Penalty for Low-Rank Representation,” NeurIPS 2011 (arXiv:1109.0367) – LADMAP. Linearizes one subproblem exactly the way Part I does; adapts the penalty from a feasibility/residual condition. **No extrapolation or momentum term anywhere** – no $2v^{k+1} - v^k$, no combination of past dual iterates.
- Xu, Figueiredo & Goldstein, “Adaptive ADMM with Spectral Penalty Parameter Selection,” AISTATS 2017 (arXiv:1605.07246) – penalty adapted each iteration via a Barzilai–Borwein spectral estimate from the primal/dual residuals. Again a residual-driven penalty update on the standard subproblems, with no extrapolation step.

Neither frames its adaptivity in terms of PDHG or Chambolle–Pock at all.

Why that is a different algorithm. Both run the *other* cut – `ladmm.py`’s own (r, x, y, δ) , with the residual taken against the stored y . Redo Part III’s bracket calculation there. The x -update’s bracket is $w^k = \gamma_k D^k - \delta^k$ with $D^k = Ax^k - y^k - d$, but the δ -update that produced δ^k used the *previous* penalty, $\gamma_{k-1} D^k = \delta^{k-1} - \delta^k$, so the two disagree by a ratio:

$$w^k = \rho_k (\delta^{k-1} - \delta^k) - \delta^k = (1 + \rho_k) v^k - \rho_k v^{k-1}, \quad \rho_k := \frac{\gamma_k}{\gamma_{k-1}}.$$

That is the PDHG step’s extrapolation only when $\rho_k = 1$. So, as best determined here: **the adaptive-LADMM scheme this literature runs is a genuinely different algorithm from adaptive PDHG whenever the penalty actually changes between iterations, and coincides with it exactly when the penalty is constant.** The fixed $(2, -1)$ was never the obstacle – the cut point was.

Closest related idea found, for honesty’s sake. Malitsky’s golden-ratio and reflected-gradient work (e.g. “Golden Ratio Algorithms for Variational Inequalities,” 2018/2020) tracks exactly this kind of issue in a related setting: its reflected/extrapolated schemes are proven equivalent to the relevant primal-dual method *under constant step sizes*, and its adaptive-step variants build the step size and the extrapolation from the same local history rather than treating them independently. So the phenomenon – adaptive steps forcing the extrapolation term to track step-size history unless the iteration is arranged to avoid it – is not unprecedented; what was not found stated anywhere is this specific comparison, for LADMM’s own common adaptive-penalty practice against Goldstein–Li–Yuan’s adaptive PDHG.